

Take Two

A rehearsal tool for science talks · Sophie Wang · github.com/sopw686/take-two

What it is

Take Two helps you practice a talk out loud. Before you rehearse, you write small notes into your script about *how* you want to say it, like "slow down on this line" or "pause here." Then you record yourself, and the app shows where you did what you planned and where you didn't.

The problem

Most speech practice tools I tried, like PowerPoint's Speaker Coach or Yoodli, grade everyone against the same idea of a good speaker: steady speed, few "ums," lots of pitch variety. That's useful, but they don't know anything about your talk. A pause right after your main result is on purpose. A pause because you lost your place isn't. These tools can't tell the two apart. And the things that go wrong in science talks are usually more specific: rushing through the one sentence with the main finding, using a technical word the audience doesn't know, or spending so long on methods that the conclusion gets squeezed in at the end.

Who it's for

Mainly grad students and researchers getting ready for conference talks, thesis defenses, or short competitions like Three Minute Thesis. Most people practice these alone, often while nervous, sometimes in a second language, and almost always with a strict time limit.

Why voice

Everything Take Two checks lives in how you say things: your speed, your pauses, your timing. If you just typed the talk out, all of that would be gone. It's also hard to notice your own pace while you're talking, because your attention is on remembering the words. So the app does the listening for you. Writing the notes and reading the results stay on a normal screen with a keyboard, since that's simply easier for those parts.

How it works

You add a few simple marks to your script: a time limit for each section, **[KEY]** for an important line you want to slow down on, / and // for short and long pauses, and **[DEFINE: term]** for a technical word you plan to explain. You read from a teleprompter-style screen while recording. Afterwards you get your script back with each mark labelled *met*, *close*, or *diverged*, the actual number behind it (for example "19% faster than your usual pace"), and a click to hear that moment. There's no overall score. There's also an Improvise mode for practicing without a script, which looks at pace, filler words, and hedges like "I think" or "kind of," compared with ranges you can set yourself.

How it's built

It's a small web app that runs on your own computer (Python with FastAPI on the back end, TypeScript in the browser). Speech-to-text is done by faster-whisper, a free model that also records when each word was said. Pauses are measured straight from the audio with Silero, a small tool that tells speech apart from silence. Speed is compared to your own average in that same recording, not to some "correct" speed. Recordings stay on your computer unless you choose to add a cloud API key. There are 133 automated tests, plus a fake recording with pauses and speed changes I know ahead of time, so I can check the whole pipeline finds them.

AI tools I used, and how

I built it with Claude Code. I wrote a spec and a short list of product rules first, then worked through it in ten steps (M1 to M10 in the commit history), reviewing and testing each one before moving on. Inside the app, Whisper and Silero handle the audio. Claude is used only for optional extras: suggesting marks for your script, checking whether you actually explained a technical term, short coaching notes based on your numbers, and content feedback in Improvise. It only ever sees text and numbers, never your audio, and the code double-checks its answers, for example throwing out any quote that doesn't actually appear in what you said. To be upfront: these Claude features have only been tested with a stand-in model so far, because I didn't have an API key on the machine I built it on.

What I'd do next

First, try it with a few real students and find out whether it actually helps them. Then run the Claude features against the real API. After that: practicing a single line at a time, coping better with paraphrasing and names the speech-to-text mishears, Q&A practice with questions drawn from your script, and importing speaker notes from PowerPoint.